

生成 AI 活用入門

明日の仕事で使えるプロンプトを、自分の手で書く 30 分

2026-07-13 / 開発部 / 30 分

30 分後、あなたは 2 つのことができるようになっていきます

一発で正解を引き当てる話ではありません。

- 自分の業務で使えるプロンプト (AI への指示文) を書いて、成果物を 1 つ出せる
- 出てきた結果の良し悪しを判断して、プロンプトを直して出し直せる

この指示では、使えるテストコードは出ません

手直しを考えると、自分で書いたほうが早い。

「このコードのテストを書いて」

- テストフレームワークが違う
- 異常系が1つも書かれていない
- モックの流儀がこのリポジトリと違う

同じモデル・同じコードで、指示の書き方だけを変えます

差を生んだのはセンスではなく、型です。

- フレームワークが合っている
- 異常系が入っている
- 手直しの土台として使える

成果物が出るプロンプトには、4つの要素が入っています

4つ全部を毎回びっしり書く必要はありません(後述)。

- 役割 — 誰として答えるか
- タスク — 何をしてほしいか
- 文脈 — 前提として何を知っておくべきか
- 出力形式 — どんな形で返すか

役割 — 誰として答えるかを一文で与えます

役割を与えると専門領域のタスクで性能が上がる。

「あなたは我々のチームのシニアエンジニアで、
テストコードのレビューを日常的にやっています」

出典: Anthropic 公式資料。

タスク — 動詞で、一度に一つだけ頼みます

もっともらしい嘘が、我々には一番たちが悪い。

- 3 つも 4 つも同時に頼むと、どれも中途半端になる
- 後輩に頼むときと同じ。一度に一つ
- 「分からないことは、分からないと言ってください」
の一文を添える

文脈 — 4 つのうち、開発部に一番効くのがここです

既存コードを 1 枚貼るだけで、出力は激変します。

- 使っているテストフレームワークは何か
- 既存のテストはどう書かれているか
- モックはどのライブラリで、どう張るか
- このリポジトリの規約は何か

出力形式 — 何に使うかを先に決めて、その形で返させます

解説付きで返されると、そのままコピーして使えません。

- そのままエディタに貼れる形で返してください
- 観点・理由・重大度の3列の表で返してください

地味ですが、毎日効いてきます。

最良のプロンプトは、最長でも最複雑でもありません

目的を確実に達成できる、最小限の構造を持つもの。

Anthropic 公式ブログより。XML タグでの構造化も、

多くの場合は不要とされています。

短く書いて、足りなければ足す。呪文の暗記は不要です。

出てきた結果は、3つの軸で見ます

なんとなく駄目、で書き直さない。

- 頼んだことが全部入っているか → 直すのはタスク
- 事実として正しいか → 直すのは文脈
- そのまま使えるか → 直すのは出力形式

直すのは、一度に一箇所だけです

一発で当てるゲームではありません。

異常系が抜けている → タスクに一行足す
「入力が空の場合と、型が違う場合を必ず含める」

3箇所まとめて直すと、
何が効いたのか分からなくなります。

今週の自分の実タスクで、プロンプトを 1 本書いてください

粗くていい。2 回目を回すところまでがゴール。

- 書く (4 分) — 4 要素を並べる。文脈に一番時間をかける
- 出す・採点する (2 分) — 3 つの評価軸で見る
- 直して出し直す (2 分) — 落ちた軸の要素を 1 つだけ

演習 (8 分)

書く → 出す → 採点 → 直す

うまくいった人は、例外なく文脈を貼っています

上手な人になったのではなく、直せる人になった。

- 役割とタスクは、皆さん自然に書けている
- 1回目で崩れるのは、たいてい文脈が薄いとき

前者は才能の話。後者は手順の話。

だから今日からできます。

貼ってはいけないものが、はっきりあります

社外との打ち合わせで、ホワイトボードに書けるか。

- 顧客の個人情報
- 認証情報・トークン
- 契約で外に出せないと決まっているコード

迷ったら、貼る前に聞いてください。

明日やることを、一つだけ決めて帰ってください

今日書いたプロンプトを、明日、別のタスクでもう一度使う。

自分の業務のプロンプトを書いて、成果物を出せる。
出力を判断して、直して、出し直せる。

効いた書き方は、チームのチャンネルへ。